Shell Scripting

Interview Notes — Linux & Bash Fundamentals

What is Shell Scripting?

Interview Definition:

“A shell script is a text file containing a sequence of Linux shell commands that are executed automatically by the shell. It is used to automate repetitive tasks and system administration activities.”

Simply:

- **Shell** = Command interpreter (like Bash)
- **Shell Script** = A file containing multiple shell commands.

Why do we use Shell Scripting?

Instead of typing the same commands repeatedly, we write them once in a script.

Example

Without a script:

```
mkdir project
cd project
touch file1.txt
touch file2.txt
ls
```

With a script (create.sh):

```
#!/bin/bash

mkdir project
cd project
touch file1.txt
touch file2.txt
ls
```

Run:

```
chmod +x create.sh
./create.sh
```

Structure of a Shell Script

```
#!/bin/bash

# Comments

echo "Hello World"
```

Shebang (#!)

```
#!/bin/bash
```

Tells Linux which interpreter should execute the script.

Basic Syntax

Print Output

```
echo "Hello"
```

Output:

```
Hello
```

Variables

```
name="Poli"

echo $name
```

Output:

```
Poli
```

User Input

```
echo "Enter your name:"
read name

echo "Welcome $name"
```

Comments

```
# This is a comment
```

If Statement

```
num=10

if [ $num -gt 5 ]
then
    echo "Greater"
fi
```

If-Else

```
read num

if [ $num -gt 0 ]
then
    echo "Positive"
else
```

```
    echo "Negative"
fi
```

If-Elif-Else

```
read marks

if [ $marks -ge 90 ]
then
    echo "A Grade"
elif [ $marks -ge 75 ]
then
    echo "B Grade"
else
    echo "C Grade"
fi
```

Loops

For Loop

```
for i in 1 2 3 4 5
do
    echo $i
done
```

Output:

```
1
2
3
4
5
```

While Loop

```
count=1

while [ $count -le 5 ]
do
    echo $count
    count=$((count+1))
done
```

Until Loop

```
count=1

until [ $count -gt 5 ]
do
    echo $count
    count=$((count+1))
done
```

Case Statement

```
read choice

case $choice in
1)
echo "Linux"
;;

2)
echo "Windows"
;;

3)
echo "Mac"
;;

*)
echo "Invalid"
;;
esac
```

Functions

```
greet()
{
    echo "Hello"
}

greet
```

With parameters

```
greet()
{
    echo "Hello $1"
}

greet Poli
```

Command Line Arguments

```
echo $1
echo $2
```

Run

```
./script.sh Hello World
```

Output:

```
Hello
World
```

Arithmetic Operations

```
a=10
b=20

sum=$((a+b))

echo $sum
```

Comparison Operators

Operator	Meaning
-eq	Equal
-ne	Not Equal
-gt	Greater Than
-lt	Less Than
-ge	Greater Than or Equal
-le	Less Than or Equal

Example

```
if [ $a -eq $b ]
```

File Test Operators

Operator	Meaning
-f	File exists
-d	Directory exists
-r	Read permission
-w	Write permission
-x	Execute permission

Example

```
if [ -f test.txt ]
then
echo "Exists"
fi
```

Exit Status

Every Linux command returns an exit status.

- 0 → Success
- Non-zero → Failure

Example

```
ls

echo $?
```

Important Special Variables

Variable	Meaning
$0	Script name
$1	First argument
$2	Second argument
$#	Number of arguments
$@	All arguments separately
$*	All arguments as one string
$$	Process ID of the script
$?	Exit status of the previous command

Real-Life Uses

- Automatic backups
- Creating users
- Monitoring disk usage
- Scheduling tasks with cron
- Log analysis
- Software installation
- Server health monitoring
- Deployment automation

Common Interview Questions

1. What is Shell Scripting?

A shell script is a file containing shell commands executed by the shell to automate tasks.

2. What is a Shell?

A shell is a command interpreter that accepts user commands and communicates with the operating system.

3. Difference between Shell and Shell Script?

Shell	Shell Script
Command interpreter	File containing shell commands
Executes one command at a time	Executes multiple commands automatically

4. What is the purpose of #!/bin/bash?

It specifies that the Bash shell should interpret and execute the script.

5. How do you make a script executable?

```
chmod +x script.sh
```

6. How do you execute a shell script?

```
./script.sh
```

or

```
bash script.sh
```

7. What is the difference between $@ and $*?

- $@ passes each argument separately (preserves individual arguments when quoted).
- $* treats all arguments as a single string when quoted.

8. How do you check if a file exists?

```
if [ -f file.txt ]
```

9. What is the difference between = and -eq?

- = is used for string comparison.
- -eq is used for numeric comparison.

10. What is $??

It stores the exit status of the last executed command (0 means success).

These are the most commonly asked shell scripting concepts and questions in Linux interviews and campus placements.